

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 7

Виконав
студент групи КН-25-1
Гордієнко В. Ю.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Алгоритми на рядках

Мета: набути практичних навичок застосування базових алгоритмів на рядках та оцінювання їх асимптотичної складності.

Порядок виконання роботи

Завдання 6. Маємо дві короткі послідовності символів: «ХУВАС» і «АВХС». Знайти найдовшу спільну підпослідовність символів, використовуючи алгоритм Хаббарда.

Розв'язання.

1. Початковий стан. Маємо першу послідовність Х, що складається з п'яти символів ХУВАС, та другу послідовність Y з чотирьох символів АВХС.

2. Будуємо числову матрицю для знаходження найдовшої спільної підпослідовності. Додаємо нульовий рядок та нульовий стовпець і заповнюємо їх нулями.

3. Рядок 1, символ Х. Зіставляємо з символами послідовності Y. Збіг відбувається лише на третьому символі Х. Відповідно, матриця в цьому рядку заповнюється значеннями: 0, 0, 0, 1, 1.

4. Рядок 2, символ Y. Збігів з послідовністю Y немає. Значення переносяться з попереднього рядка: 0, 0, 0, 1, 1.

5. Рядок 3, символ В. Збіг відбувається на другому символі. Починаючи з цієї позиції значення оновлюються. Поточний рядок матриці: 0, 0, 1, 1, 1.

6. Рядок 4, символ А. Збіг відбувається на першому символі. Поточний рядок матриці: 0, 1, 1, 1, 1.

7. Рядок 5, символ С. Збіг відбувається на четвертому символі. Оскільки перед цим максимальне накопичене значення було 1, тепер воно стає 2. Поточний рядок матриці: 0, 1, 1, 1, 2.

8. Максимальне значення в останній клітинці дорівнює 2, що означає довжину найдовшої спільної підпослідовності.

9. Для відновлення самої підпослідовності здійснюємо зворотний хід. Оскільки в процесі побудови максимуми дублювалися, алгоритм передбачає

декілька альтернативних шляхів збирання результату.

10.Перший можливий варіант зворотного обходу збирає символи X та C, утворюючи підпоследовність XC.

11.Другий можливий варіант зворотного обходу збирає символи B та C, утворюючи підпоследовність BC.

12.Третій можливий варіант зворотного обходу збирає символи A та C, утворюючи підпоследовність AC.

13.Отже, результуючі найдовші спільні підпоследовності мають довжину 2 і це: XC, BC, AC.

L[i][j]	Ø (0)	A (1)	B (2)	X (3)	C (4)
Ø (0)	0	0	0	0	0
X (1)	0	0	0	1	1
Y (2)	0	0	0	1	1
B (3)	0	0	1	1	1
A (4)	0	1	1	1	1
C (5)	0	1	1	1	2

Таблиця 1 – Матриця до завдання 6

Висновки: на цій практичній роботі я набув практичних навичок застосування базових алгоритмів на рядках та оцінювання їх асимптотичної складності.

Контрольні питання

1. У чому полягає задача знаходження найдовшої спільної підпоследовності (LCS)?

Пошук найдовшої последовності символів, яка є спільною для двох рядків. При цьому символи не обов'язково мають іти підряд, але їхній відносний порядок має зберігатися (наприклад, для ABCDE та ACEB результатом є ACE).

2. Які головні методи можна використовувати для знаходження найдовшої спільної підпослідовності?

Повний перебір всіх комбінацій (неефективний брутфорс), класичне динамічне програмування (DP) та оптимізовані позиційні алгоритми (наприклад, алгоритм Хаббарда / Ханта-Шимані чи методи на основі бітових паралелізмів).

3. Як працює алгоритм динамічного програмування для знаходження LCS?

Будується матриця розміром $n * m$, де при збігу символів до значення по діагоналі додається 1, а при розбіжності – береться максимум із сусідніх комірок (зверху або ліворуч); результат знаходиться в останній комірці, а сама послідовність відновлюється зворотним ходом.

4. Як працює алгоритм Хаббарда для знаходження LCS?

Замість прорахунку всієї матриці, алгоритм знаходить лише координати символів, що збігаються (match pairs), а потім шукає серед них найдовшу зростаючу підпослідовність (LIS) за допомогою швидкого бінарного пошуку.

5. Які переваги та недоліки алгоритмів динамічного програмування та Хаббарда для знаходження LCS?

Динамічне програмування працює стабільно на будь-яких даних, але витрачає забагато пам'яті $O(n * m)$ під матрицю. Алгоритм Хаббарда надшвидкий і економний на розріджених даних (коли збігів мало), але в найгіршому випадку (при масових повтореннях символів) стає значно повільнішим за DP.

6. Які існують практичні застосування для задачі знаходження найдовшої спільної підпослідовності?

Порівняння молекул ДНК та РНК у біоінформатиці, відстеження змін у коді через команду `git diff`, порівняння текстових файлів в утилітах типу `diff`, а також виявлення запозичень в антиплагіат-системах.